

Mentor AI Full Project Guide

1. What This Product Is

Mentor AI is a personalized career growth platform for students. The app should answer three core questions for every user:

- â€¢ Where am I today?
- â€¢ Where do I want to go?
- â€¢ What should I do next?

The product becomes valuable over weeks and months because it combines user profile data, resume data, progress tracking, roadmap planning, interview preparation, and AI chat into one system.

2. Current State Of This Repo

This repository is still at the scaffold stage.

Backend status

- â€¢ Register flow exists.
- â€¢ Prisma is configured for PostgreSQL.
- â€¢ JWT generation exists.
- â€¢ Password hashing exists.
- â€¢ Only one API route is implemented: auth register.
- â€¢ Only one database model exists: User.

Frontend status

- â€¢ App routing exists.
- â€¢ Layout shells exist.
- â€¢ Pages are mostly placeholders.
- â€¢ No real dashboard data flow exists yet.
- â€¢ No real auth flow exists yet.

file, so the app can route correctly.

3. Product Modules

Build the system as separate modules so the code stays maintainable.

Backend modules

- â€¢ Auth
- â€¢ User
- â€¢ Profile
- â€¢ Resume
- â€¢ Skills
- â€¢ Goals
- â€¢ Roadmap
- â€¢ Projects
- â€¢ Interview
- â€¢ Analytics
- â€¢ AI
- â€¢ Chat
- â€¢ Admin
- â€¢ Shared utilities

Frontend pages

- â€¢ Landing
- â€¢ Login
- â€¢ Register
- â€¢ Dashboard
- â€¢ Profile
- â€¢ Resume
- â€¢ Roadmap
- â€¢ Projects
- â€¢ Interview Prep
- â€¢ AI Chat
- â€¢ Analytics
- â€¢ Settings
- â€¢ Admin

4. Recommended Folder Structure

Use a clean modular structure

Backend folder structure

```
server/
|-- prisma/
|   |-- schema.prisma
|   |-- migrations/
|
|-- src/
|   |-- app.js
|   |-- server.js
|   |
|   |-- config/
|   |   |-- db.js
|   |   |-- env.js
|   |   |-- cors.js
|   |
|   |-- controllers/
|   |   |-- auth.controller.js
|   |   |-- user.controller.js
|   |   |-- profile.controller.js
|   |   |-- resume.controller.js
|   |   |-- goal.controller.js
|   |   |-- roadmap.controller.js
|   |   |-- project.controller.js
|   |   |-- interview.controller.js
|   |   |-- analytics.controller.js
|   |   |-- ai.controller.js
|   |   |-- chat.controller.js
|   |   |-- admin.controller.js
|   |
|   |-- routes/
|   |   |-- auth.routes.js
|   |   |-- user.routes.js
|   |   |-- profile.routes.js
|   |   |-- resume.routes.js
|   |   |-- goal.routes.js
|   |   |-- roadmap.routes.js
|   |   |-- project.routes.js
|   |   |-- interview.routes.js
|   |   |-- analytics.routes.js
|   |   |-- ai.routes.js
|   |   |-- chat.routes.js
|   |   |-- admin.routes.js
|   |
|   |-- services/
|   |   |-- auth.services.js
|   |   |-- user.services.js
|   |   |-- profile.services.js
|   |   |-- resume.services.js
|   |   |-- goal.services.js
|   |   |-- roadmap.services.js
|   |   |-- project.services.js
|   |   |-- interview.services.js
```

```

| | |-- analytics.services.js
| | |-- ai.services.js
| | |-- chat.services.js
| | |-- admin.services.js
| |
| |-- validators/
| | |-- auth.validator.js
| | |-- user.validator.js
| | |-- profile.validator.js
| | |-- resume.validator.js
| | |-- goal.validator.js
| | |-- roadmap.validator.js
| | |-- project.validator.js
| | |-- interview.validator.js
| | |-- analytics.validator.js
| | |-- ai.validator.js
| | |-- chat.validator.js
| | |-- admin.validator.js
| |
| |-- middleware/
| | |-- auth.middleware.js
| | |-- error.middleware.js
| | |-- async.middleware.js
| | |-- role.middleware.js
| | |-- rateLimit.middleware.js
| |
| |-- utils/
| | |-- generateToken.js
| | |-- hashToken.js
| | |-- apiResponse.js
| | |-- AppError.js
| | |-- constants.js
| | |-- logger.js
| |
| |-- jobs/
| | |-- resumeParse.job.js
| | |-- aiGenerate.job.js
| |
| |-- integrations/
| | |-- gemini.js
| | |-- cloundinary.js
| | |-- imagekit.js
| | |-- redis.js
| |
| |-- tests/
| | |-- auth.test.js
| | |-- profile.test.js
| | |-- resume.test.js
| | |-- roadmap.test.js
| | |-- ai.test.js

```

Frontend folder structure

client/

```

|-- src/
|   |-- main.jsx
|   |-- App.jsx
|   |-- routes/
|   |   |-- AppRoutes.jsx
|   |   |-- ProtectedRoute.jsx
|   |   |-- PublicRoute.jsx
|   |
|   |-- pages/
|   |   |-- Landing/
|   |   |-- Login/
|   |   |-- Register/
|   |   |-- Dashboard/
|   |   |-- Profile/
|   |   |-- Resume/
|   |   |-- Roadmap/
|   |   |-- Projects/
|   |   |-- Interview/
|   |   |-- AIChat/
|   |   |-- Analytics/
|   |   |-- Settings/
|   |   |-- Admin/
|   |
|   |-- components/
|   |   |-- auth/
|   |   |-- dashboard/
|   |   |-- profile/
|   |   |-- resume/
|   |   |-- roadmap/
|   |   |-- interview/
|   |   |-- analytics/
|   |   |-- chat/
|   |   |-- admin/
|   |   |-- common/
|   |
|   |-- context/
|   |-- hooks/
|   |-- layouts/
|   |-- services/
|   |-- styles/
|   |-- utils/
|   |-- assets/

```

5. Database Design

Use PostgreSQL with Prisma.

Core entities

â€¢ User
â€¢ Profile

- â€¢ Resume
- â€¢ Skill
- â€¢ Goal
- â€¢ Roadmap
- â€¢ RoadmapItem
- â€¢ ProjectRecommendation
- â€¢ InterviewSession
- â€¢ InterviewAnswer
- â€¢ ChatMessage
- â€¢ ProgressEvent
- â€¢ AnalyticsSnapshot
- â€¢ ApiUsage
- â€¢ AdminAuditLog
- â€¢ RefreshToken or Session

Suggested relationships

- â€¢ User to Profile: one to one
- â€¢ User to Resume: one to many
- â€¢ User to Skill: one to many or many to many
- â€¢ User to Goal: one to many
- â€¢ User to Roadmap: one to many
- â€¢ Roadmap to RoadmapItem: one to many
- â€¢ User to ProjectRecommendation: one to many
- â€¢ User to InterviewSession: one to many
- â€¢ InterviewSession to InterviewAnswer: one to many
- â€¢ User to ChatMessage: one to many
- â€¢ User to ProgressEvent: one to many

Important design rule

Do not put all user progress inside a single JSON blob. Use relational tables for production reporting, filtering, and analytics.

6. Authentication Plan

Build authentication first because every other feature depends on it.

Required auth features

- â€¢ Register
- â€¢ Login

- â€ Refresh token
- â€ Forgot password
- â€ Reset password
- â€ Email verification later

Auth production rules

- â€ Hash passwords with bcrypt.
- â€ Store access tokens separately from refresh tokens.
- â€ Use short-lived access tokens.
- â€ Store refresh tokens in a revocable way.
- â€ Add role-based authorization.
- â€ Rate limit auth endpoints.
- â€ Validate all inputs with Zod.

Auth API examples

- â€ POST /api/v1/auth/register
- â€ POST /api/v1/auth/login
- â€ POST /api/v1/auth/logout
- â€ POST /api/v1/auth/refresh-token
- â€ POST /api/v1/auth/forgot-password
- â€ POST /api/v1/auth/reset-password
- â€ GET /api/v1/auth/me

7. Profile Module

The profile is the foundation for personalization.

Store these fields

- â€ Name
- â€ College
- â€ Degree
- â€ Graduation year
- â€ Skills
- â€ Interests
- â€ Preferred role
- â€ GitHub
- â€ LinkedIn
- â€ Resume reference

Profile logic

- â€¢ Add completion percentage.
- â€¢ Track which fields are missing.
- â€¢ Validate social URLs.
- â€¢ Allow editing after onboarding.

Profile API examples

- â€¢ GET /api/v1/profile/me
- â€¢ POST /api/v1/profile
- â€¢ PATCH /api/v1/profile
- â€¢ GET /api/v1/profile/completion

8. Resume Module

This feature must be reliable and asynchronous.

Resume pipeline

- â€¢ User uploads PDF.
- â€¢ Server validates file type and size.
- â€¢ File is uploaded to Cloudinary or ImageKit.
- â€¢ Metadata is stored in PostgreSQL.
- â€¢ Text is extracted from the PDF.
- â€¢ Text is cleaned and normalized.
- â€¢ AI summaries and skill extraction are generated.
- â€¢ Structured fields are saved.

Resume storage fields

- â€¢ File URL
- â€¢ Provider
- â€¢ Original filename
- â€¢ File size
- â€¢ MIME type
- â€¢ Extracted text
- â€¢ Parsed skills
- â€¢ Experience summary
- â€¢ Project summary

Resume API examples

- â€¢ POST /api/v1/resumes/upload
- â€¢ GET /api/v1/resumes/me
- â€¢ GET /api/v1/resumes/:id
- â€¢ DELETE /api/v1/resumes/:id
- â€¢ POST /api/v1/resumes/:id/reprocess

9. AI Analysis Module

This is the main differentiator.

What the AI should analyze

- â€¢ Skills
- â€¢ Resume
- â€¢ Projects
- â€¢ Experience
- â€¢ Current role goal
- â€¢ Roadmap progress
- â€¢ Interview performance

Expected AI output structure

- â€¢ Current Level
- â€¢ Strengths
- â€¢ Weaknesses
- â€¢ Missing Skills
- â€¢ Resume Suggestions
- â€¢ Projects to Build
- â€¢ Interview Readiness
- â€¢ Estimated Readiness Score

Production AI rules

- â€¢ Never send only the raw user prompt.
- â€¢ Always send user context from the database.
- â€¢ Force structured JSON output.
- â€¢ Monitor and log all AI outputs.

â€¢ Run long AI jobs in background workers.

AI API examples

â€¢ POST /api/v1/ai/analyze-career
â€¢ POST /api/v1/ai/generate-roadmap
â€¢ POST /api/v1/ai/recommend-projects
â€¢ POST /api/v1/ai/review-interview-answer
â€¢ POST /api/v1/ai/mentor-chat

10. Roadmap Module

Roadmaps should be personalized and versioned.

Roadmap behavior

â€¢ Use the user profile and skill gaps.
â€¢ Generate weekly learning blocks.
â€¢ Break goals into tasks.
â€¢ Track progress per task.
â€¢ Allow regeneration without deleting history.

Example roadmap layout

â€¢ Week 1: React Router, Context API
â€¢ Week 2: Express, PostgreSQL
â€¢ Week 3: Authentication and JWT
â€¢ Week 4: Build project and deploy

Roadmap API examples

â€¢ POST /api/v1/roadmaps/generate
â€¢ GET /api/v1/roadmaps/me
â€¢ GET /api/v1/roadmaps/:id
â€¢ PATCH /api/v1/roadmaps/items/:id/complete
â€¢ PATCH /api/v1/roadmaps/items/:id/reschedule

11. Goals Module

Goal types

- â€¢ Learning goals
- â€¢ Interview goals
- â€¢ Project goals
- â€¢ Revision goals
- â€¢ Habit goals

Goal fields

- â€¢ Title
- â€¢ Description
- â€¢ Due date
- â€¢ Status
- â€¢ Priority
- â€¢ Completion date
- â€¢ Source type
- â€¢ Source module

Goal API examples

- â€¢ POST /api/v1/goals
- â€¢ GET /api/v1/goals
- â€¢ PATCH /api/v1/goals/:id
- â€¢ PATCH /api/v1/goals/:id/complete
- â€¢ DELETE /api/v1/goals/:id

12. Project Recommendation Module

Projects should depend on the chosen role and current gaps.

Example backend developer projects

- â€¢ URL Shortener
- â€¢ Expense Tracker
- â€¢ Chat App
- â€¢ Auth API project
- â€¢ Task manager

- â€¢ Difficulty
- â€¢ Estimated time
- â€¢ Required skills
- â€¢ Why it fits the user
- â€¢ GitHub examples or inspiration links

Project API examples

- â€¢ GET /api/v1/projects/recommendations
- â€¢ POST /api/v1/projects/generate
- â€¢ GET /api/v1/projects/:id
- â€¢ PATCH /api/v1/projects/:id/save

13. Interview Preparation Module

Track user improvement over time.

Interview sections

- â€¢ HR questions
- â€¢ Technical questions
- â€¢ Behavioral questions
- â€¢ Role-specific questions

Interview data to store

- â€¢ Question
- â€¢ User answer
- â€¢ AI feedback
- â€¢ Score
- â€¢ Attempt date
- â€¢ Topic category

Interview API examples

- â€¢ POST /api/v1/interviews/generate
- â€¢ GET /api/v1/interviews
- â€¢ GET /api/v1/interviews/:id

14. Skill Tracker Module

The dashboard should show skill growth over time.

Store

- â€ Skill name
- â€ Category
- â€ Current level
- â€ Target level
- â€ Evidence source
- â€ Last updated date

Display examples

- â€ React
- â€ Node
- â€ SQL
- â€ DSA
- â€ System design

Skill API examples

- â€ GET /api/v1/skills
- â€ POST /api/v1/skills
- â€ PATCH /api/v1/skills/:id
- â€ DELETE /api/v1/skills/:id

15. Analytics Module

Analytics must be based on real database events.

Analytics to show

- â€ Weekly study hours

- â€¢ Roadmap progress
- â€¢ AI usage
- â€¢ Token usage

Production analytics rules

- â€¢ Store raw events.
- â€¢ Precompute summaries if needed.
- â€¢ Index query columns.
- â€¢ Keep expensive queries out of request paths.

Analytics API examples

- â€¢ GET /api/v1/analytics/dashboard
- â€¢ GET /api/v1/analytics/study-hours
- â€¢ GET /api/v1/analytics/goals
- â€¢ GET /api/v1/analytics/interview-score
- â€¢ GET /api/v1/analytics/skills

16. AI Chat Mentor

This should not be generic chat.

Chat behavior

Before answering, the system should read:

- â€¢ Profile
- â€¢ Resume summary
- â€¢ Roadmap progress
- â€¢ Goal state
- â€¢ Skill gaps
- â€¢ Recent interviews

Chat storage

- â€¢ User message
- â€¢ Assistant message
- â€¢ Context snapshot

Chat API examples

- â€¢ POST /api/v1/chat/message
- â€¢ GET /api/v1/chat/history
- â€¢ DELETE /api/v1/chat/history/:id

17. Admin Module

Admin features should be isolated and protected.

Admin dashboard should show

- â€¢ Users
- â€¢ Daily active users
- â€¢ API usage
- â€¢ Reports generated
- â€¢ AI token usage
- â€¢ Failed jobs
- â€¢ Audit logs

Admin API examples

- â€¢ GET /api/v1/admin/overview
- â€¢ GET /api/v1/admin/users
- â€¢ GET /api/v1/admin/usage
- â€¢ GET /api/v1/admin/reports
- â€¢ GET /api/v1/admin/audit-logs

18. Shared Production Infrastructure

Environment variables

- â€¢ DATABASE_URL
- â€¢ JWT_SECRET
- â€¢ JWT_REFRESH_SECRET
- â€¢ CLIENT_URL

- â€¢ IMAGEKIT_PUBLIC_KEY
- â€¢ IMAGEKIT_PRIVATE_KEY
- â€¢ GEMINI_API_KEY
- â€¢ REDIS_URL
- â€¢ SMTP_HOST
- â€¢ SMTP_PORT
- â€¢ SMTP_USER
- â€¢ SMTP_PASS
- â€¢ NODE_ENV
- â€¢ PORT

Infrastructure recommendations

- â€¢ Use Redis for caching and rate limits.
- â€¢ Use a background queue for PDF parsing and AI generation.
- â€¢ Use Cloudinary or ImageKit for file storage.
- â€¢ Use PostgreSQL as the source of truth.
- â€¢ Use structured logs and request IDs.
- â€¢ Use separate environments for development, staging, and production.

19. Validation And Testing

What to test

- â€¢ Validation schemas
- â€¢ Auth flows
- â€¢ Resume upload
- â€¢ AI JSON parsing
- â€¢ Roadmap generation
- â€¢ Goal completion
- â€¢ Interview scoring
- â€¢ Role-based access
- â€¢ Admin protection

Testing layers

- â€¢ Unit tests for services
- â€¢ Integration tests for controllers and routes
- â€¢ Schema validation tests
- â€¢ E2E tests for key user journeys

Minimum production setup

- â€¢ Dockerize backend and frontend
- â€¢ Use a managed PostgreSQL database
- â€¢ Use managed object storage or media storage
- â€¢ Use environment-based secrets
- â€¢ Add CI checks for lint and tests
- â€¢ Add migration execution in deploy pipeline

Recommended release stages

- â€¢ Local development
- â€¢ Auth MVP
- â€¢ Profile and resume
- â€¢ AI analysis
- â€¢ Roadmap and goals
- â€¢ Interview and analytics
- â€¢ Admin and hardening
- â€¢ Production deploy

21. Step-By-Step Build Order For A New Developer

Phase 1

- â€¢ Set up env validation.
- â€¢ Fix auth completely.
- â€¢ Add error handling.
- â€¢ Add versioned routes.

Phase 2

- â€¢ Add profile and resume schemas.
- â€¢ Add upload and extraction flow.
- â€¢ Store extracted data.

Phase 3

Phase 4

- â€¢ Add roadmap, goals, and progress tracking.
- â€¢ Make dashboards data-driven.

Phase 5

- â€¢ Add project recommendations.
- â€¢ Add interview preparation.
- â€¢ Add chat mentor.

Phase 6

- â€¢ Add analytics.
- â€¢ Add admin.
- â€¢ Add tests, observability, queueing, and deployment.

22. Notes For The Current Repo

The current codebase already gives you:

- â€¢ Express server bootstrap
- â€¢ Prisma client setup
- â€¢ JWT helper
- â€¢ Password hashing
- â€¢ Auth registration skeleton
- â€¢ Frontend routing skeleton
- â€¢ Placeholder UI components

The current codebase does not yet give you:

- â€¢ Login
- â€¢ Refresh tokens
- â€¢ Password reset
- â€¢ Profile data model
- â€¢ Resume upload pipeline
- â€¢ AI analysis service
- â€¢ Roadmap engine
- â€¢ Goal tracking
- â€¢ Analytics backend

23. Final Advice

Build this as a product platform, not a collection of AI prompts.

The database should be the memory layer. The AI should be a service that reads from that memory layer. The dashboard should visualize progress over time. The roadmap should update from user behavior. The chat should be personalized from stored state.

That is what turns Mentor AI from a demo into a production system.